The ZUC-256 Stream Cipher

Abstract. In this paper, we describe the ZUC-256 stream cipher, a successor of the previous ZUC-128 stream cipher used in the 3GPP confidentiality and integrity algorithms 128-EEA3 and 128-EIA3. The aim is a new stream cipher that offers the 256-bit security for the upcoming applications in 5G. For the authentication, various tag sizes are supported with the IV-respecting restriction.

Keywords: Stream ciphers, ZUC, 256-bit security.

1 Introduction

The core of the 3GPP confidentiality and integrity algorithms 128-EEA3 and 128-EIA3 is the ZUC-128 stream cipher [1]. With the development of the communication and computing technology, there is an emerging need for the new core stream cipher in the upcoming 5G applications which offers 256-bit security. To be highly compatible with the current 128-bit version, we present the ZUC-256 stream cipher, which is a successor of the previous ZUC-128 stream cipher. The new ZUC-256 stream cipher differs from ZUC-128 only in the initialization phase and in the message authentication codes (MAC) generation phase, other aspects are all the same as the previous ZUC-128 algorithm.

This paper is structured as follows. In Section 2, we give the detailed description of the new ZUC-256 stream cipher, including both the initialization phase, the keystream generation phase and the MAC generation phase. Finally, some conclusions are drawn in Section 3.

2 The Description of the Cipher

In this section, we will present the detailed description of the ZUC-256 stream cipher. The following notations will be used hereafter.

- Denote the integer modular addition by $\boxplus$, i.e., for $0 \leq x < 2^{32}$ and $0 \leq y < 2^{32}$, $x \boxplus y$ is the integer addition mod 2^{32}.
- Denote the integer addition modulo $2^{31} - 1$ by $x + y$ mod $2^{31} - 1$ for $1 \leq x \leq 2^{31} - 1$ and $1 \leq y \leq 2^{31} - 1$.
- Denote the bitwise exclusive OR by $\oplus$.
- Denote the bit string concatenation by $\|$.
- Denote the bitwise logic OR by $|$.
- $K = (K_{31}, K_{30}, ..., K_2, K_1, K_0)$, the 256-bit secret key used in the ZUC-256 where K_i for $0 \leq i \leq 31$ are 8-bit bytes.

- $IV = (IV_{24}, IV_{23}, ..., IV_{17}, IV_{16}, IV_{15}, \ldots, IV_1, IV_0)$, the 184-bit initialization vector used in the ZUC-256 where IV_i for $0 \leq i \leq 16$ are 8-bit bytes and IV_i for $17 \leq i \leq 24$ are 6-bit string occupying the 6 least significant bits of a byte.
- d_i for $0 \leq i \leq 15$ are the 7-bit constants used in the ZUC-256 stream cipher.
- $\lll$, the left rotation of a 64-bit operand, $x \lll n$ means $((x \ll n) \mid (x \gg (64-n)))$.

As depicted in Fig.1 and Fig.2, there are 3 parts involved in ZUC-256: a 496-bit linear feedback shift register (LFSR) defined over the field GF($2^{31}-1$), consisting of 16 31-bit cells $(s_{15}, s_{14}, \cdots, s_2, s_1, s_0)$ defined over the set $\{1, 2, \cdots, 2^{31}-1\}$; a bit reorganization layer (BR), which extracts the content of the LFSR to form 4 32-bit words, (X_0, X_1, X_2, X_3), used in the following finite state machine (FSM); there are 2 32-bit words R_1 and R_2 used as the memory in the FSM.

The Key/IV loading scheme is as follows.

$$
\begin{aligned}
s_0 &= K_0 \parallel d_0 \parallel K_{21} \parallel K_{16} \\
s_1 &= K_1 \parallel d_1 \parallel K_{22} \parallel K_{17} \\
s_2 &= K_2 \parallel d_2 \parallel K_{23} \parallel K_{18} \\
s_3 &= K_3 \parallel d_3 \parallel K_{24} \parallel K_{19} \\
s_4 &= K_4 \parallel d_4 \parallel K_{25} \parallel K_{20} \\
s_5 &= IV_0 \parallel (d_5 \mid IV_{17}) \parallel K_5 \parallel K_{26} \\
s_6 &= IV_1 \parallel (d_6 \mid IV_{18}) \parallel K_6 \parallel K_{27} \\
s_7 &= IV_{10} \parallel (d_7 \mid IV_{19}) \parallel K_7 \parallel IV_2 \\
s_8 &= K_8 \parallel (d_8 \mid IV_{20}) \parallel IV_3 \parallel IV_{11} \\
s_9 &= K_9 \parallel (d_9 \mid IV_{21}) \parallel IV_{12} \parallel IV_4 \\
s_{10} &= IV_5 \parallel (d_{10} \mid IV_{22}) \parallel K_{10} \parallel K_{28} \\
s_{11} &= K_{11} \parallel (d_{11} \mid IV_{23}) \parallel IV_6 \parallel IV_{13} \\
s_{12} &= K_{12} \parallel (d_{12} \mid IV_{24}) \parallel IV_7 \parallel IV_{14} \\
s_{13} &= K_{13} \parallel d_{13} \parallel IV_{15} \parallel IV_8 \\
s_{14} &= K_{14} \parallel (d_{14} \mid (K_{31})_H^4) \parallel IV_{16} \parallel IV_9 \\
s_{15} &= K_{15} \parallel (d_{15} \mid (K_{31})_L^4) \parallel K_{30} \parallel K_{29},
\end{aligned}
$$

where $(K_{31})_H^4$ is the high 4 bits of the byte K_{31} and $(K_{31})_L^4$ is the low 4 bits of K_{31}, and the constants d_i for $0 \leq i \leq 15$ are defined as follows.

$$
\begin{aligned}
d_0 &= 0100010 \\
d_1 &= 0101111 \\
d_2 &= 0100100 \\
d_3 &= 0101010
\end{aligned}
$$

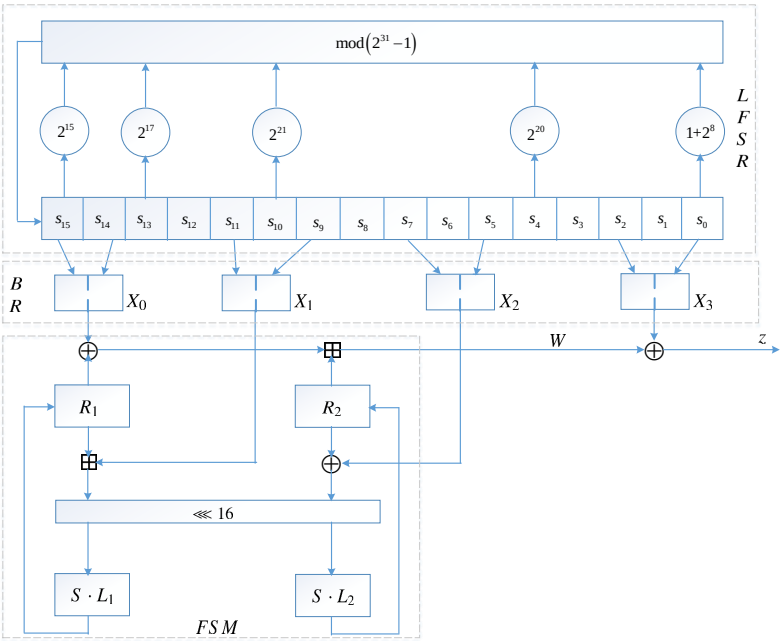

Fig. 1. The keystream generation phase of the ZUC-256 stream cipher

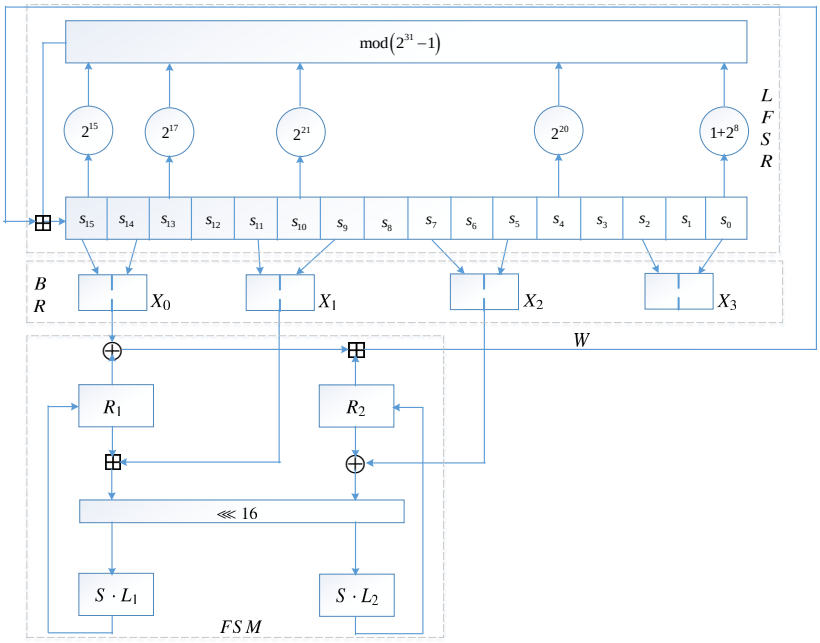

Fig. 2. The initialization phase of the ZUC-256 stream cipher

$$
\begin{aligned}
d_4 &= 1101101 \\
d_5 &= 1000000 \\
d_6 &= 1000000 \\
d_7 &= 1000000 \\
d_8 &= 1000000 \\
d_9 &= 1000000 \\
d_{10} &= 1000000 \\
d_{11} &= 1000000 \\
d_{12} &= 1000000 \\
d_{13} &= 1010010 \\
d_{14} &= 0010000 \\
d_{15} &= 0110000.
\end{aligned}
$$

There are $32 + 1 = 33$ rounds of initialization in the ZUC-256, which is depicted as follows.

1. Load the key, IV and constants into the LFSR as specified above.
2. Let $R_1 = R_2 = 0$.
3. for $i = 0$ to 31 do
   - Bitreorganization( )
   - $Z = F(X_0, X_1, X_2)$
   - LFSRWithInitializationMode($Z \gg 1$)
4. - Bitreorganization( )
   - $Z = F(X_0, X_1, X_2)$ and discard Z
   - LFSRWithworkMode().

Now we specify the relevant subroutines one-by-one.

LFSRWithInitializationMode(u)

1. $v = 2^{15} \cdot s_{15} + 2^{17} \cdot s_{13} + 2^{21} \cdot s_{10} + 2^{20} \cdot s_4 + (1 + 2^8) \cdot s_0 \mod(2^{31} - 1)$
2. if $v = 0$ then set $v = 2^{31} - 1$
3. $s_{16} = v + u \bmod(2^{31} - 1)$
4. if $s_{16} = 0$ then set $s_{16} = 2^{31} - 1$
5. $(s_{16}, s_{15}, \cdots, s_2, s_1) \rightarrow (s_{15}, s_{14}, \cdots, s_1, s_0)$.

LFSRWithworkMode()

1. $s_{16} = 2^{15} \cdot s_{15} + 2^{17} \cdot s_{13} + 2^{21} \cdot s_{10} + 2^{20} \cdot s_4 + (1 + 2^8) \cdot s_0 \mod(2^{31} - 1)$
2. if $s_{16} = 0$ then set $s_{16} = 2^{31} - 1$
3. $(s_{16}, s_{15}, \cdots, s_2, s_1) \rightarrow (s_{15}, s_{14}, \cdots, s_1, s_0)$.

Bitreorganization()

1. $X_0 = s_{15H} \parallel s_{14L}$
2. $X_1 = s_{11L} \parallel s_{9H}$

3. $X_2 = s_{7L} \parallel s_{5H}$
4. $X_3 = s_{2L} \parallel s_{0H}$,

where s_{iH} is the high 16 bits of the cell s_i and s_{jL} is the low 16 bits of the cell s_j.

$F(X_0, X_1, X_2)$

1. $W = (X_0 \oplus R_1) \boxplus R_2$
2. $W_1 = R_1 \boxplus X_1$
3. $W_2 = R_2 \oplus X_2$
4. $R_1 = S(L_1(W_{1L} \parallel W_{2H}))$
5. $R_2 = S(L_2(W_{2L} \parallel W_{1H}))$,

where $S = (S_0, S_1, S_0, S_1)$ is the 4 parallel S-boxes which are the same as those used in the previous ZUC-128 and L_1 and L_2 are the two MDS matrices used in the ZUC-128. The ZUC-256 stream cipher generates a 32-bit keystream word at each time instant.

KeystreamGeneration()

1. Bitreorganization( )
2. $Z = F(X_0, X_1, X_2) \oplus X_3$
3. LFSRWithworkMode().

ZUC-256 generates 20000-bit keystream for each frame, i.e., for each frame it produces 625 keystream words; after that a key/IV re-synchronization is performed with the key/constants fixed and the IV changing into a new value.

The MAC generation algorithm of ZUC-256 is as follows. Let $M = (m_0, m_1, \cdots, m_{l-1})$ be the l-bit length plaintext message and the size t of the tag is selectively to be of 32, 64 and 128 bits.

MAC_Generation(M)

1. Let ZUC-256 produce a keystream of $L = \lceil \frac{l}{32} \rceil + 2 \cdot \frac{t}{32}$ words. Denote the keystream bit string by $z_0, z_1, \cdots, z_{32 \cdot L-1}$, where z_0 is the most significant bit of the first output keystream word and z_{31} is the least significant bit of the keystream word.
2. Initialize $Tag = (z_0, z_1, \cdots, z_{t-1})$
3. for $i = 0$ to $l - 1$ do
   - let $W_i = (z_{t+i}, \cdots, z_{i+2t-1})$
   - if $m_i = 1$ then $Tag = Tag \oplus W_i$
4. $W_l = (z_{l+t}, \cdots, z_{l+2t-1})$
5. $Tag = Tag \oplus W_l$
6. return Tag

For the different sizes of the MAC tag, to prevent the forgery attack, the constants are specified as follows.

1. for the tag size of 32 bits, the constants are

$$
\begin{aligned}
d_0 &= 0100010 \\
d_1 &= 0101111 \\
d_2 &= 0100101 \\
d_3 &= 0101010 \\
d_4 &= 1101101 \\
d_5 &= 1000000 \\
d_6 &= 1000000 \\
d_7 &= 1000000 \\
d_8 &= 1000000 \\
d_9 &= 1000000 \\
d_{10} &= 1000000 \\
d_{11} &= 1000000 \\
d_{12} &= 1000000 \\
d_{13} &= 1010010 \\
d_{14} &= 0010000 \\
d_{15} &= 0110000.
\end{aligned}
$$

2. for the tag size of 64 bits, the constants are

$$
\begin{aligned}
d_0 &= 0100011 \\
d_1 &= 0101111 \\
d_2 &= 0100100 \\
d_3 &= 0101010 \\
d_4 &= 1101101 \\
d_5 &= 1000000 \\
d_6 &= 1000000 \\
d_7 &= 1000000 \\
d_8 &= 1000000 \\
d_9 &= 1000000 \\
d_{10} &= 1000000 \\
d_{11} &= 1000000 \\
d_{12} &= 1000000 \\
d_{13} &= 1010010 \\
d_{14} &= 0010000 \\
d_{15} &= 0110000.
\end{aligned}
$$

3. for the tag size of 128 bits, the constants are

$$\begin{aligned}
d_0 &= 0100011\\
d_1 &= 0101111\\
d_2 &= 0100101\\
d_3 &= 0101010\\
d_4 &= 1101101\\
d_5 &= 1000000\\
d_6 &= 1000000\\
d_7 &= 1000000\\
d_8 &= 1000000\\
d_9 &= 1000000\\
d_{10} &= 1000000\\
d_{11} &= 1000000\\
d_{12} &= 1000000\\
d_{13} &= 1010010\\
d_{14} &= 0010000\\
d_{15} &= 0110000.
\end{aligned}$$

The test vectors of the ZUC-256 stream cipher for the keystream generation phase are as follows.

1. let $K_i =$ `0x00` for $0 \leq i \leq 31$ and $IV_i =$ `0x00` for $0 \leq i \leq 24$, then the first 20 keystream words are
   - `58d03ad6,2e032ce2,dafc683a,39bdcb03,52a2bc67,`
   - `f1b7de74,163ce3a1,01ef5558,9639d75b,95fa681b,`
   - `7f090df7,56391ccc,903b7612,744d544c,17bc3fad,`
   - `8b163b08,21787c0b,97775bb8,4943c6bb,e8ad8afd`
2. let $K_i =$ `0xff` for $0 \leq i \leq 31$ and $IV_i =$ `0xff` for $0 \leq i \leq 16$ and $IV_i =$ `0x3f` for $17 \leq i \leq 24$, then the first 20 keystream words are
   - `3356cbae,d1a1c18b,6baa4ffe,343f777c,9e15128f,`
   - `251ab65b,949f7b26,ef7157f2,96dd2fa9,df95e3ee,`
   - `7a5be02e,c32ba585,505af316,c2f9ded2,7cdbd935,`
   - `e441ce11,15fd0a80,bb7aef67,68989416,b8fac8c2`

The test vectors of the ZUC-256 stream cipher for the tag authentication phase are as follows.

1. let $K_i =$ `0x00` for $0 \leq i \leq 31$ and $IV_i =$ `0x00` for $0 \leq i \leq 24$, $M = 0x\underbrace{00, \cdots, 00}_{100}$ with the length $l = 400$-bit, then the 32-bit tag, 64-bit tag and 128-bit tag are
   - The 32-bit mac is `9b972a74`

- The 64-bit mac is `673e5499 0034d38c`
- The 128-bit mac is `d85e54bb cb960096 7084c952 a1654b26`

2. let $K_i =$ `0x00` for $0 \leq i \leq 31$ and $IV_i =$ `0x00` for $0 \leq i \leq 24$, $M = 0x\underbrace{11, \cdots, 11}_{1000}$ with the length $l = 4000$-bit, then the 32-bit tag, 64-bit tag and 128-bit tag are
   - The 32-bit mac is `8754f5cf`
   - The 64-bit mac is `130dc225 e72240cc`
   - The 128-bit mac is `df1e8307 b31cc62b eca1ac6f 8190c22f`
3. let $K_i =$ `0xff` for $0 \leq i \leq 31$ and $IV_i =$ `0xff` for $0 \leq i \leq 16$ and $IV_i =$ `0x3f` for $17 \leq i \leq 24$, $M = 0x\underbrace{00, \cdots, 00}_{100}$ with the length $l = 400$-bit, then the 32-bit tag, 64-bit tag and 128-bit tag are
   - The 32-bit mac is `1f3079b4`
   - The 64-bit mac is `8c71394d 39957725`
   - The 128-bit mac is `a35bb274 b567c48b 28319f11 1af34fbd`
4. let $K_i =$ `0xff` for $0 \leq i \leq 31$ and $IV_i =$ `0xff` for $0 \leq i \leq 16$ and $IV_i =$ `0x3f` for $17 \leq i \leq 24$, $M = 0x\underbrace{11, \cdots, 11}_{1000}$ with the length $l = 4000$-bit, then the 32-bit tag, 64-bit tag and 128-bit tag are
   - The 32-bit mac is `5c7c8b88`
   - The 64-bit mac is `ea1dee54 4bb6223b`
   - The 128-bit mac is `3a83b554 be408ca5 494124ed 9d473205`

The security claim of the ZUC-256 stream cipher is the 256-bit security in the 5G application setting. For the forgery attacks on the authentication part, the security level is the same as the tag size and the IV is not allowed to be re-used. If the tag verification failed, no output should be generated.

3 Conclusions

In this paper, we have presented the details of the new ZUC-256 stream cipher. Any cryptanalysis is welcome.

References

1. Specification of the 3GPP Confidentiality and Integrity Algorithms 128-EEA3 and 128-EIA3, Document 4: Design and Evaluation Reprot. `http://www.gsmworld.com/documents/EEA3_EIA3_Design_Evaluation_v1_1.pdf`.